

NAME: COLLINS AMADI  
STUDENT ID: FWSD2511357  
STUDENT EMAIL: [fwsd2511357@students.icdfa.edu.ng](mailto:fwsd2511357@students.icdfa.edu.ng)  
COURSE CODE: SBT-DF203  
LAB NAME: ARP POISONING FORENSICS  
DATE: 15/09/2026  
INSTRUCTOR: AMINU IDRIS

#### Important Pre-Lab Instruction

Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.

|                    |                                                               |
|--------------------|---------------------------------------------------------------|
| Course Code        | SBT-DF203                                                     |
| Course Title       | Basic Networking Skills for Digital Forensics                 |
| Lab Number         | Lab 5                                                         |
| Lab Title          | ARP Poisoning Forensics                                       |
| Delivery Block     | 2/3 of 3                                                      |
| Scheduled Dates    | 12-18 September 2026                                          |
| Estimated Duration | 4-5 hours practical work plus report writing                  |
| Mode               | Individual practical lab in an authorized virtual environment |
| Required Evidence  | arp.pcap and an isolated host-only ARP capture                |

## 1. Source Materials and Slide Alignment

- Address Resolution Protocol Forensics: ARP operation, ARP caches, request/reply fields, capture with tshark, ARP spoofing/poisoning, man-in-the-middle consequences and evidence analysis.
- The slide practical compares clean and poisoned ARP tables and uses a supplied arp.py script only within the same LAN.
- The assignment requires legitimate ARP capture, spoofing capture and proof of a poisoned ARP table.

#### Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

## 2. Lab Scenario

A user reports that the default gateway intermittently resolves to the wrong MAC address. You will first document normal ARP resolution, then analyze a known ARP-poisoning capture. In an instructor-approved host-only network, you may reproduce a short controlled poisoning event, capture the evidence, restore the ARP mappings and explain how the attack is detected.

## 3. Learning Outcomes

- Explain how ARP maps IPv4 addresses to MAC addresses on a LAN.
- Inspect ARP caches on Linux and Windows systems.
- Identify ARP requests, legitimate replies and unsolicited or conflicting replies.
- Detect one IP address being associated with multiple MAC addresses.
- Capture and document a bounded ARP-poisoning simulation in a host-only network.
- Restore network state and recommend preventive controls.

## 4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.

- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The live component is optional and must use instructor-assigned RFC1918 addresses on a host-only/internal virtual switch. Never poison a home, school, office, hotel or public Wi-Fi network.

## 5. Required Tools and Environment

| Item             | Recommended Tool                                   | Purpose                                |
|------------------|----------------------------------------------------|----------------------------------------|
| Virtual machines | Kali attacker/analyst plus Linux or Windows victim | Create an isolated LAN scenario.       |
| Network mode     | Host-only/internal network                         | Prevent access to production networks. |
| Capture tools    | Wireshark and tshark                               | Capture ARP packets.                   |
| Packet library   | python3-scapy                                      | Run authorized training scripts.       |
| System tools     | ip neigh, arp, ipconfig /all                       | Inspect ARP tables and interfaces.     |
| Evidence         | arp.pcap or student capture                        | Analyze known poisoning indicators.    |

## 6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y wireshark tshark python3-scapy net-tools
wget -O evidence/arp.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-
  lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_table_initial.txt
```

Screenshot required: Interface addresses, route/gateway, initial ARP table and evidence hashes.

**ANSWER:**

```
(collins@kali)~/SBT-DF203-Lab5
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(collins@kali)~/SBT-DF203-Lab5
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(collins@kali)~/SBT-DF203-Lab5
$ ip -br address | tee reports/interfaces.txt
lo        UNKNOWN    127.0.0.1/8 ::1/128
eth0      UP            192.168.188.130/24 fe80::20c:29ff:fec3:2528/64

(collins@kali)~/SBT-DF203-Lab5
$ ip route | tee reports/routes.txt
default via 192.168.188.2 dev eth0 proto dhcp src 192.168.188.130 metric 100
192.168.188.0/24 dev eth0 proto kernel scope link src 192.168.188.130 metric 100

(collins@kali)~/SBT-DF203-Lab5
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.188.254 dev eth0 lladdr 00:50:56:f5:f7:90 STALE
192.168.188.2 dev eth0 lladdr 00:50:56:e8:6a:f6 STALE
```

## 7. Mini Evidence and Chain-of-Custody Worksheet

| Field                 | Student Entry                |
|-----------------------|------------------------------|
| Case/lab identifier   | SBT-DF203-Lab5-COLLINS AMADI |
| Trainee name          | COLLINS AMADI                |
| Date and time started | 15/09/2026 03:19:39          |

| Field                       | Student Entry                                                                                                                                                                         |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Evidence file name(s)       | evidence/arp.pcap                                                                                                                                                                     |
| Source or generation method | KALI LINUX COMMAND                                                                                                                                                                    |
| Original SHA-256            | 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e<br>evidence/arp.pcap<br>342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e<br>working/arp_working.pcap |
| Working-copy SHA-256        | working/arp_working.pcap                                                                                                                                                              |
| Analysis workstation/VM     | COLLINS@KALI                                                                                                                                                                          |
| Notes on any changes        |                                                                                                                                                                                       |

## 8. Part A - Observe Normal ARP Resolution

1. Identify the victim IP, gateway IP and your interface name.
2. Clear only the specific training neighbor entry where permitted.
3. Start an ARP-only capture, ping the gateway once and stop the capture.
4. Identify the broadcast request and unicast reply.

```

IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
sleep 2
ping -c 1 "$GATEWAY_IP"
wait
ip neigh show | tee reports/arp_table_after_ping.txt

```

```

(collins@kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"

(collins@kali)-[~/SBT-DF203-Lab5]
$ sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
[1] 32064

(collins@kali)-[~/SBT-DF203-Lab5]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("evidence/normal_arp.pcapng") could not be opened: Permission denied.

[1] + exit 1 sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng
(collins@kali)-[~/SBT-DF203-Lab5]
$ sleep 2

(collins@kali)-[~/SBT-DF203-Lab5]
$ ping -c 1 "$GATEWAY_IP"
PING 192.168.188.2 (192.168.188.2) 56(84) bytes of data.
64 bytes from 192.168.188.2: icmp_seq=1 ttl=128 time=0.590 ms

--- 192.168.188.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.590/0.590/0.590/0.000 ms

(collins@kali)-[~/SBT-DF203-Lab5]
$ wait

(collins@kali)-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_after_ping.txt
192.168.188.2 dev eth0 lladdr 00:50:56:e8:6a:f6 REACHABLE

```

ANSWER:

## 9. Part B - Analyze ARP Request and Reply Fields

```

tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv

```

| Field                   | Normal Request     | Normal Reply                |
|-------------------------|--------------------|-----------------------------|
| Ethernet destination    | ff:ff:ff:ff:ff:ff  | Requester MAC               |
| ARP opcode              | 1                  | 2                           |
| Sender protocol address | Requester IP       | Target IP                   |
| Sender hardware address | Requester MAC      | Target MAC                  |
| Target protocol address | Target IP          | Requester IP                |
| Forensic interpretation | Who has target IP? | Target IP is at target MAC. |

## ANSWER:

```
(collins@kali)~/SBT-DF203-Lab5
$ PCAP=working/arp_working.pcap

(collins@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:
fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:
65

(collins@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(collins@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv
4      2023-03-12T04:22:44.904123642+0100      136.160.215.194 00:50:56:86:02:65      00:50:56:86:cb:fc
6      2023-03-12T04:22:49.943043799+0100      136.160.215.15 00:50:56:86:cb:fc      00:50:56:86:02:65

(collins@kali)~/SBT-DF203-Lab5
$
```

## 10. Part C - Analyze the Supplied Poisoning Capture

```
PCAP=working/arp_working.pcap
# Inventory all ARP claims
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv

# Summarize IP-to-MAC claims
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt

# Look for gratuitous or unsolicited-looking replies
tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv
```

## ANSWER:

```
(collins@kali)~/SBT-DF203-Lab5
$ PCAP=working/arp_working.pcap

(collins@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:
fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:
65

(collins@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(collins@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv
4      2023-03-12T04:22:44.904123642+0100      136.160.215.194 00:50:56:86:02:65      00:50:56:86:cb:fc
6      2023-03-12T04:22:49.943043799+0100      136.160.215.15 00:50:56:86:cb:fc      00:50:56:86:02:65

(collins@kali)~/SBT-DF203-Lab5
$
```

## 11. Part D - Optional Controlled Host-Only Simulation

### Instructor Approval Required

Skip this section unless the virtual network is confirmed as host-only/internal and the victim, gateway and analyst IP addresses are assigned specifically for the lab.

- Record clean ARP tables on victim and gateway.
- Start ARP capture on the analyst VM.
- Run the instructor-provided arp.py for no more than 30 seconds.
- Capture the changed victim mapping and stop the script.
- Restore correct ARP mappings or restart the isolated VMs.

# Example placeholders only - replace with instructor-assigned lab IPs

VICTIM\_IP=192.168.56.20

GATEWAY\_IP=192.168.56.1

IFACE=eth0

```
sudo tshark -i "$IFACE" -f 'arp' -a duration:40 -w evidence/controlled_arp_poison.pcapng &
sleep 2
```

# In another terminal, run the instructor-provided script for <=30 seconds:

```
# sudo timeout 30 python3 scripts/arp.py "$VICTIM_IP" "$GATEWAY_IP"
```

wait

```
ip neigh show | tee reports/analyst_arp_table_after.txt
```

## 12. Part E - Detection Logic and Timeline

| Time     | Claimed IP      | Claimed MAC       | Target            | Request Seen First? | Assessment |
|----------|-----------------|-------------------|-------------------|---------------------|------------|
| 04:22:44 | 136.160.215.194 | 00:50:56:86:02:65 | 00:50:56:86:cb:fc |                     |            |
| 04:22:49 | 136.160.215.15  | 00:50:56:86:cb:fc | 00:50:56:86:02:65 |                     |            |
|          | 192.168.188.254 | 00:50:56:f5:f7:90 | 00:50:56:e8:6a:f6 | STALE               |            |
|          | 192.168.188.2   | 00:50:56:e8:6a:f6 | 00:50:56:f5:f7:90 | STALE               |            |

- Flag a gateway IP that changes from its legitimate MAC to the analyst/attacker MAC.
- Flag repeated ARP replies without a corresponding recent request.
- Flag one MAC claiming both victim and gateway IP addresses.
- Correlate ARP evidence with packet forwarding, DNS/HTTP anomalies and endpoint ARP-cache changes.

## 13. Restoration and Prevention

# Stop any training script first, then clear learned entries on the lab VM

```
sudo ip neigh flush all
```

# Renew/reconnect the isolated interface as appropriate

```
ip neigh show
```

# Confirm no ARP-poisoning process remains

```
ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt
```

- Use Dynamic ARP Inspection with DHCP snooping on managed switches where available.
- Segment networks, enable port security and monitor gateway IP-to-MAC consistency.
- Use encrypted application protocols so interception does not expose plaintext.
- Deploy ARP monitoring tools and correlate alerts with switch and endpoint logs.

### ANSWER:

```
(collins@kali)~/SBT-DF203-Lab5
$ ip neigh show | tee reports/analyst_arp_table_after.txt
192.168.188.254 dev eth0 lladdr 00:50:56:f5:f7:90 STALE
192.168.188.2 dev eth0 lladdr 00:50:56:e8:6a:f6 STALE

(collins@kali)~/SBT-DF203-Lab5
$ sudo ip neigh flush all

(collins@kali)~/SBT-DF203-Lab5
$ ip neigh show

(collins@kali)~/SBT-DF203-Lab5
$ ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt

(collins@kali)~/SBT-DF203-Lab5
$
```